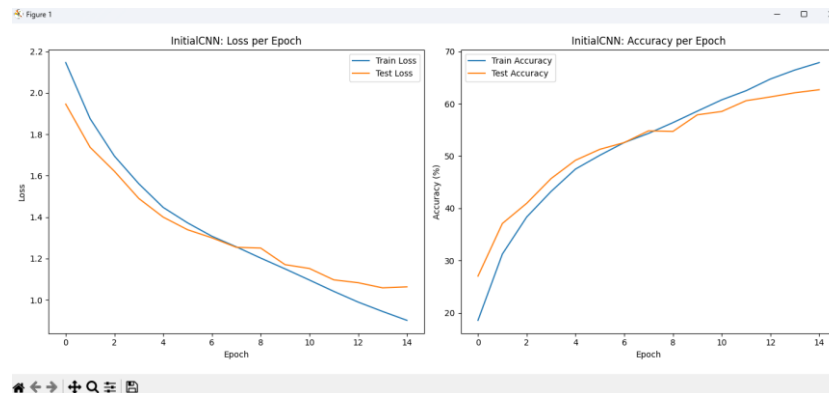


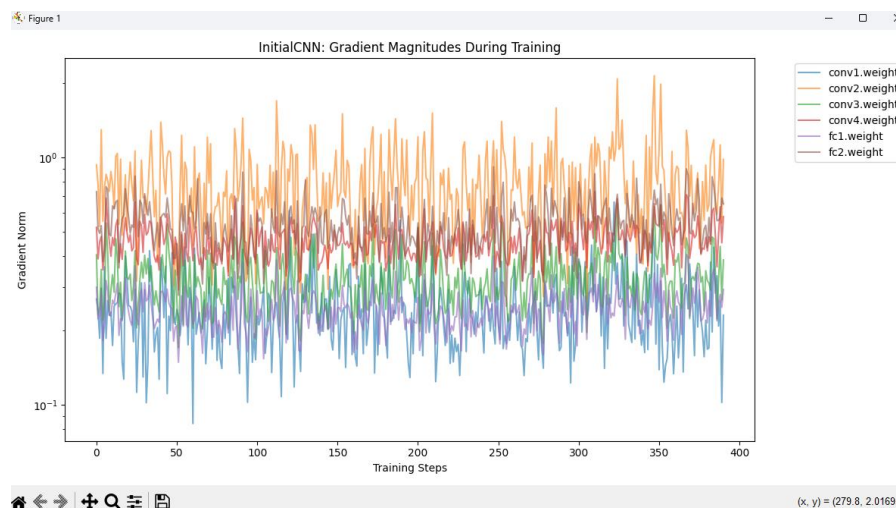
Github URL: <https://github.com/adiYossef/Task2>

Submission by: adi yossef – 213507494, ghofran abuliel - 213566232

Section 1:



In the plot we can see that the train loss and the test loss are both going down as we do more epochs, which mean that the model succeed to learn. We can also see this from the right plot, where the accuracy of the model is rising as we do more epochs.



In this plot, we can see a very interesting phenomenon, the vanishing gradient. Y-axis represents the magnitude of the gradients being applied to the weights of our network (we plot this in a logarithmic scale so this is why it jumps from 10^{-1} to 10^0).

X-axis represents the number of batches that our model has processed during the training.

When we look at the graph we can see that the layers closest to the output have relatively large gradient norms, and as we move backward through the network, the gradients get smaller and smaller, until we get to the first layer which has the smallest gradients of all, hovering around 10^{-1} .

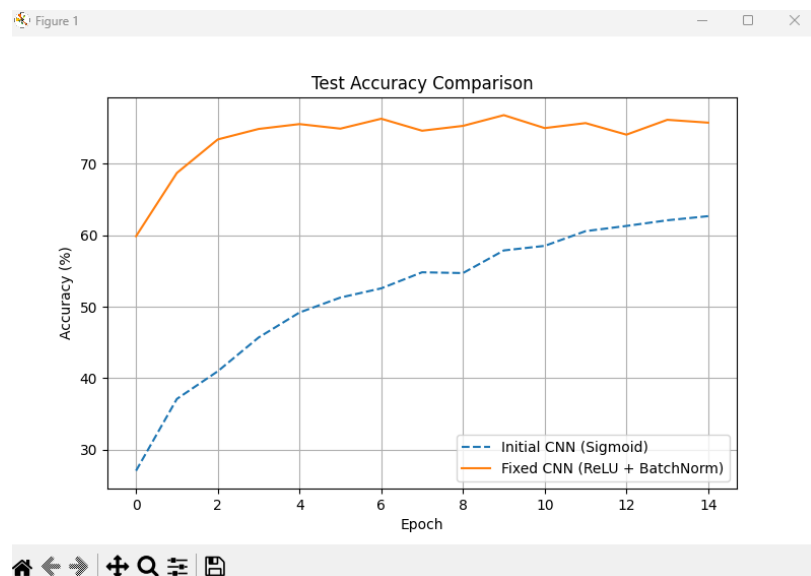
The problem

The InitialCNN uses `torch.sigmoid` as its activation function. The derivative of a sigmoid function is at most 0.25. When backpropagation computes gradients for earlier layers (like `conv1`), it multiplies these small derivatives together repeatedly. Because the network is deep (4 convolutional layers and 2 linear layers), the gradients shrink to near-zero before they can update the earlier weights. Consequently, the network fails to learn effectively, which you will see reflected in flat or poor learning curves.

The fix

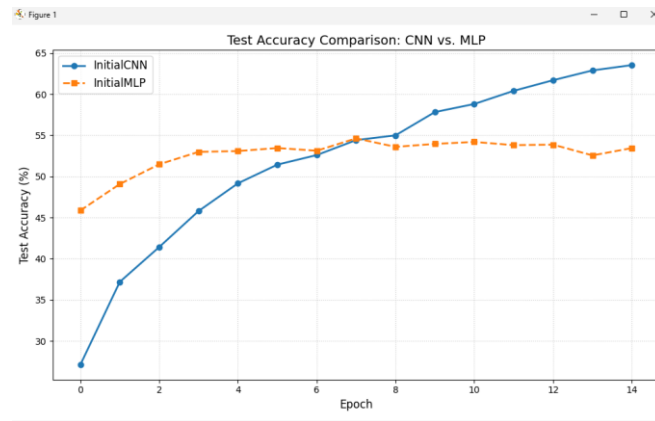
To fix this, we will swap the sigmoid activations for ReLU (which does not squash the positive gradients) and add a single **Batch Normalization** layer to fulfill the "one additional layer" and "normalization" requirements.

And now for the comparison between the two models:



As we can see that the fixed CNN has more accuracy than the Initial CNN.

Section 2:



From this plot we can see that the CNN is outperforming the MLP. This is happening because the MLP treats the image as a flat 1D vector and it destroys the spatial relationships whereas the CNN preserve the 2D grid structure, allowing them to learn local patterns regardless of where they appear in the image.

If we look at the parameters of the models we can see that the MLP has vastly more parameters. The first layer alone connects $32 \times 32 \times 3$ inputs which is 3072, to 512 neurons, resulting in over 1.5 million weights just for fc1. On the other hand, the CNN uses weight sharing, keeping the parameter count incredibly low while maintaining deeper architecture.

You can see the comparison between the parameters of each model:

InitialCNN Parameters: 653,386

InitialMLP Parameters: 1,707,274

Because the MLP has over a million weights, it requires more memory to store the model architecture and gradients.

The CNN requires less memory for the weights themselves and this is why it consumes less memory.

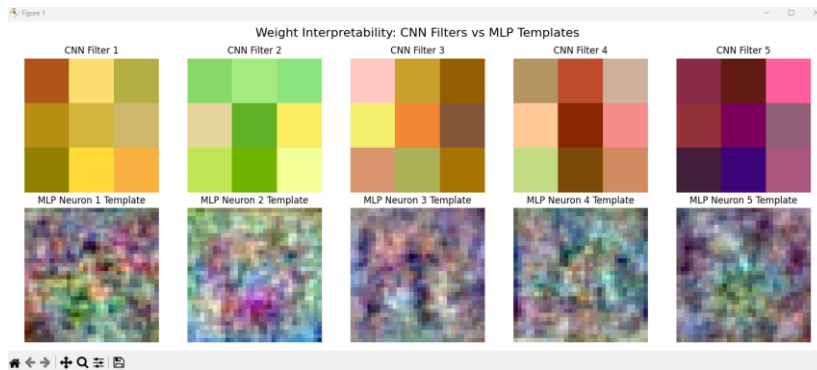
InitialCNN Memory Size: 2.492 MB

InitialMLP Memory Size: 6.513 MB

All that talk but the runtime speed for the MLP is much faster than the runtime speed of the CNN, and this is happening because the MLP only do dense matrix multiplication while the CNN does convolutions which is more expensive mathematically and this is why it takes more time for each epoch.

MLP Training Time: 118.28 seconds

CNN Training Time: 165.41 seconds



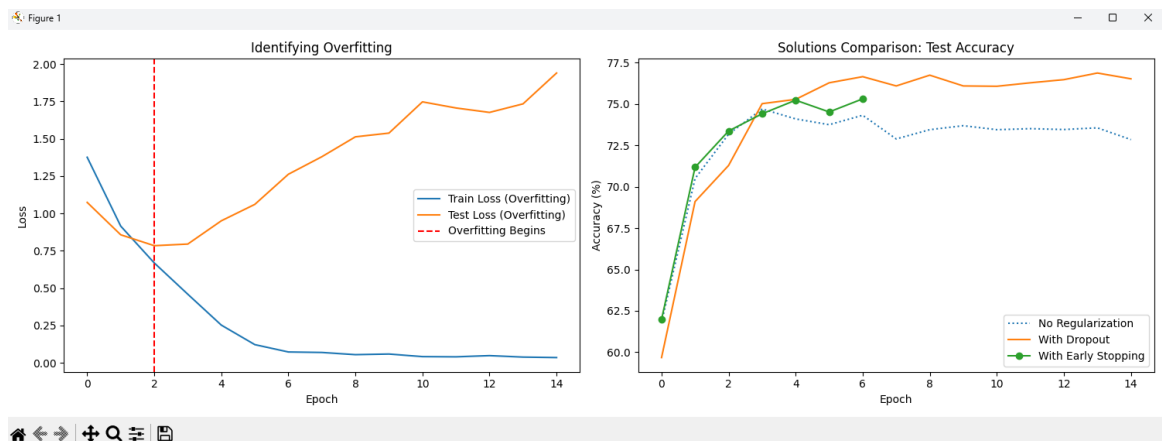
From this plot we can see how each model is learning.

The first row shows the optimized weights of the very first layer of the CNN, while the second row shows the weights from the first hidden layer of the MLP, which have been reshaped back into a 32x32 format so we can view them as images. We can notice that the CNN learn small features in each grid while the MLP instead try to learn all the features at once.

CNNs are best for data with a known grid-like topology where spatial locality matters.

NLPs are best for unstructured data, where each feature has a distinct fixed meaning, and there is no spatial relationship between the columns.

Section 3:



From the left plot we can see that we identify the overfitting from epoch 2, that means that the loss of the training will get smaller and smaller because the model just memorizing the dataset but the loss on the test from this epoch won't get down and only get bigger, and we can see that this is actually what is happening.

We tested two methods to reduce the overfitting, dropout and early stopping. The dropout method randomly zeroes out neurons during training, and this

prevents the network from relying too heavily on any single pathway or "memorizing" specific features, forcing it to learn more robust and redundant representations.

The early stopping monitors the validation loss. Once the validation loss degrades for a set number of epochs, training is terminated and this locks the model weights at their point of optimal point.

We can see from the right plot that the accuracy of both the dropout and the early stopping is much higher than the accuracy of the overfitting model, which mean that both methods succeed to overcome the overfitting problem.

Section 4:

Part 4 - Data Augmentation

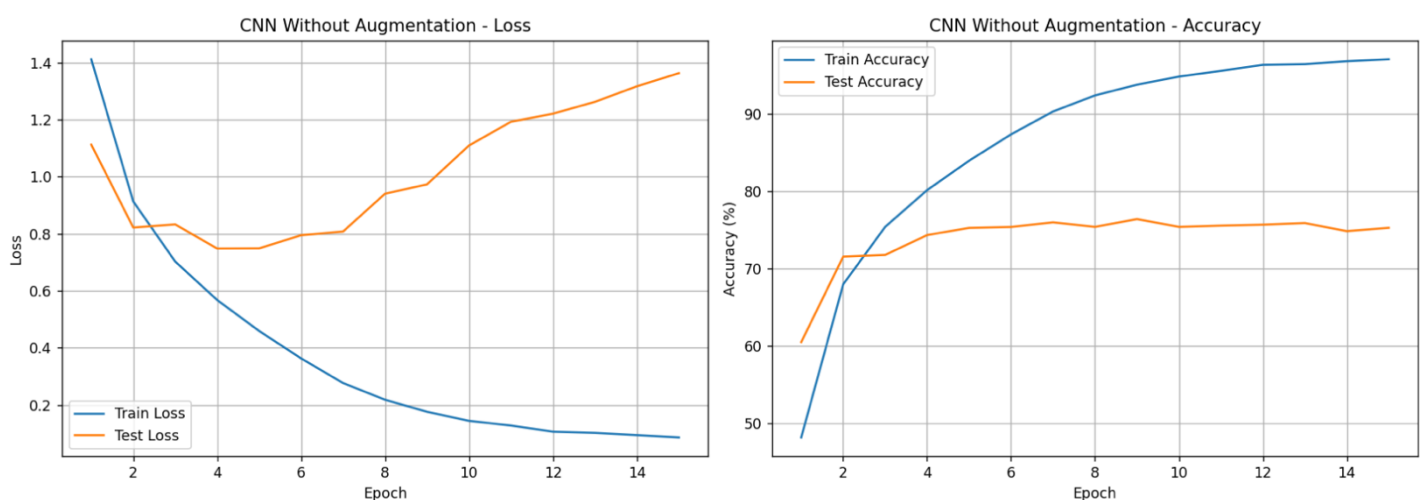
Several augmentation techniques were applied to the CIFAR-10 training set, including random crop, rotation, zoom/resize, horizontal flip, and vertical flip. These transformations generate additional image variations and increase the diversity of the training data. The CNN was trained both with and without augmentation, and the results were compared using accuracy curves, loss curves, and confusion matrices.



Examples of the applied augmentation techniques, including random crop, rotation, zoom/resize, horizontal flip, and vertical flip.

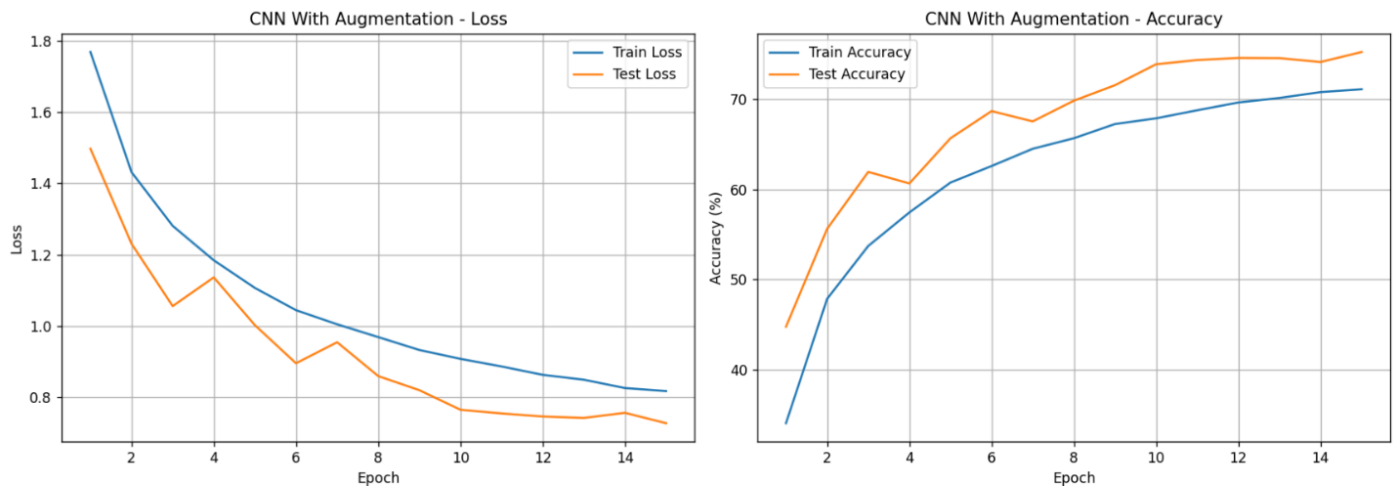
CNN Without Augmentation

The CNN was first trained on the original CIFAR-10 dataset without any augmentation. The model achieved a final test accuracy of 75.27% after 15 epochs. Training accuracy increased continuously and reached 97.02%, while test accuracy stabilized around 75%, indicating significant overfitting. The loss curves show that the training loss kept decreasing, whereas the test loss increased after several epochs, further confirming that the model began memorizing the training data.



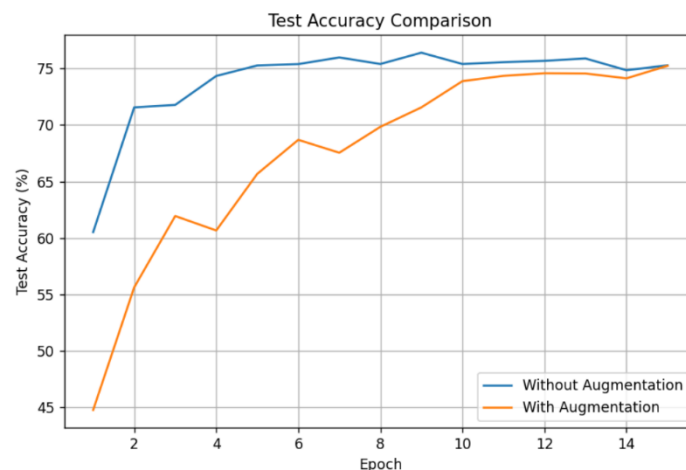
CNN With Augmentation

The CNN was then trained using the augmented dataset. The augmentation techniques included random crop, rotation, zoom/resize, horizontal flip, and vertical flip. After 15 epochs, the model achieved a final test accuracy of 75.23%. Although the training process was slower, both training and test losses decreased steadily. The gap between training and test accuracy remained much smaller than in the non-augmented model, indicating better generalization and reduced overfitting.



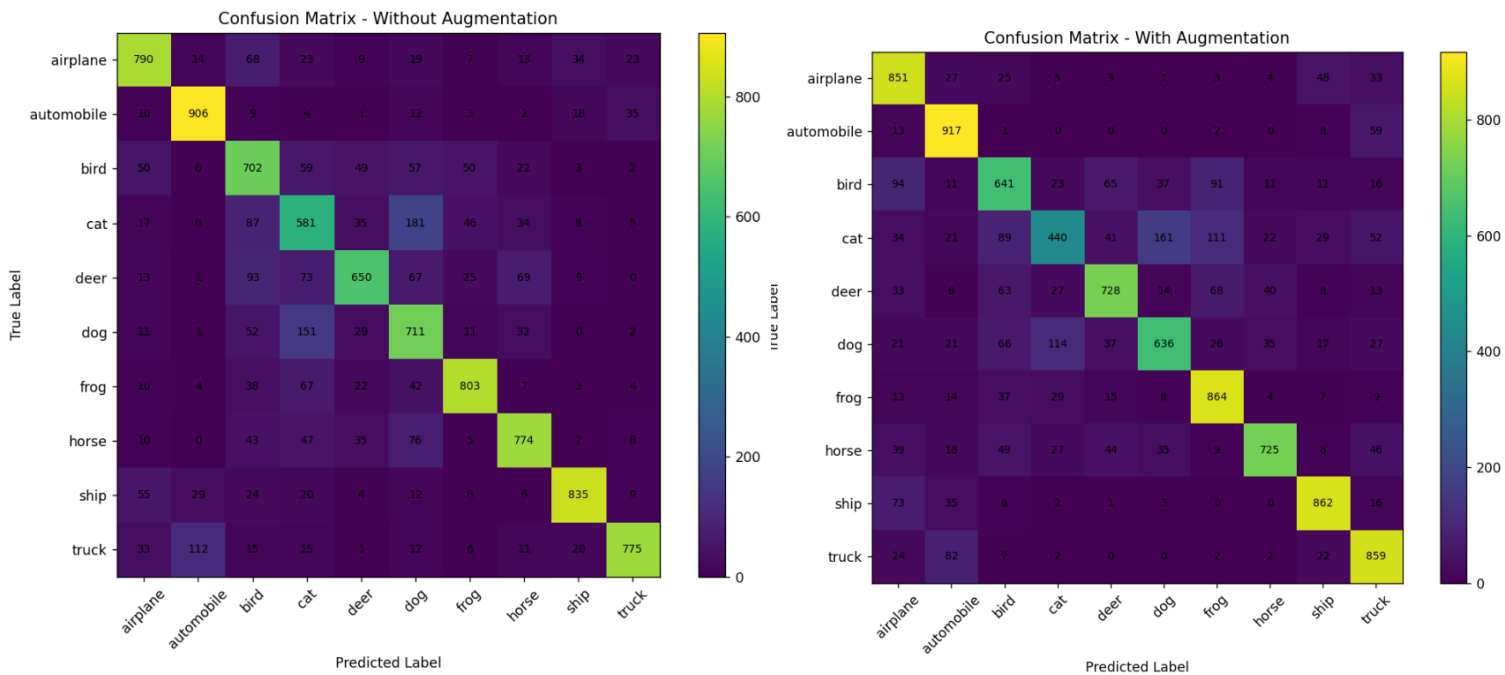
Accuracy Comparison

Both models achieved very similar final test accuracy. The non-augmented model reached 75.27%, while the augmented model achieved 75.23%. Although augmentation did not significantly improve the final accuracy, it reduced overfitting and produced a more stable learning process. The augmented model continued improving throughout training, while the non-augmented model plateaued after several epochs.



Confusion Matrix Analysis

The confusion matrices show that both models performed well on classes such as automobile, airplane, frog, ship, and truck. Most classification errors occurred between visually similar classes, particularly cats and dogs. The augmented model achieved comparable classification performance while maintaining better generalization, confirming the observations from the accuracy and loss curves.



Conclusion

In this task, data augmentation techniques including random crop, rotation, zoom/resize, horizontal flip, and vertical flip were applied to the CIFAR-10 dataset. The CNN trained without augmentation achieved a final test accuracy of 75.27% with a training time of 1437.23 seconds, while the augmented CNN achieved a final test accuracy of 75.23% with a training time of 1642.77 seconds. Although augmentation slightly increased the training time and produced a similar final accuracy, it reduced overfitting and improved the model's generalization ability. These results demonstrate that data augmentation helps the model learn more robust features and improves its performance on unseen data.

```
Final Summary
-----
Without Augmentation Final Test Accuracy: 75.27%
With Augmentation Final Test Accuracy: 75.23%
Without Augmentation Training Time: 1437.23 seconds
With Augmentation Training Time: 1642.77 seconds
```

Part 5 - Creativity task

Intuition Behind the Model

The proposed hybrid architecture combines the strengths of Convolutional Neural Networks (CNNs) and Multi-Layer Perceptrons (MLPs). The CNN component is responsible for extracting meaningful spatial features from the input images, such as edges, textures, and local patterns. These extracted features are then passed to an MLP classifier, which learns higher-level relationships between the features and performs the final classification. The goal of this design is to benefit from the feature extraction capabilities of CNNs while leveraging the classification power of MLPs.

Training Results

The hybrid model was trained for 15 epochs on the CIFAR-10 dataset. The training curves show a consistent decrease in both training and testing loss throughout the training process. Similarly, the accuracy curves indicate continuous improvement in classification performance, reaching a best test accuracy of 79.97%. Although a moderate gap between training and testing accuracy appeared during later epochs, the model maintained strong generalization performance and achieved the highest accuracy among all tested models.

```
Hybrid CNN-MLP Model
-----
Number of trainable parameters: 1,276,682
Estimated model memory size: 4.870 MB
```

```
--- Training Hybrid CNN-MLP ---
```

```
Epoch 1/15
Train Loss: 1.3180
Train Accuracy: 51.60%
Test Loss: 1.0592
Test Accuracy: 62.28%
-----
```

```
Epoch 2/15
Train Loss: 0.9461
Train Accuracy: 66.61%
Test Loss: 0.8484
Test Accuracy: 70.70%
-----
```

```
Epoch 3/15
Train Loss: 0.8098
Train Accuracy: 71.85%
Test Loss: 0.8677
Test Accuracy: 70.52%
-----
```

```
Epoch 4/15
Train Loss: 0.7197
Train Accuracy: 74.98%
Test Loss: 0.7489
Test Accuracy: 74.01%
-----
```

```
Epoch 5/15
Train Loss: 0.6509
Train Accuracy: 77.30%
Test Loss: 0.6800
Test Accuracy: 76.52%
-----
```

```
Epoch 6/15
Train Loss: 0.5903
Train Accuracy: 79.49%
Test Loss: 0.7397
Test Accuracy: 74.55%
-----
```

```
Epoch 7/15
Train Loss: 0.5436
Train Accuracy: 81.12%
Test Loss: 0.6684
Test Accuracy: 77.34%
-----
```

```
Epoch 8/15
Train Loss: 0.4899
Train Accuracy: 82.97%
Test Loss: 0.7650
Test Accuracy: 75.07%
-----
```

```
Epoch 9/15
Train Loss: 0.4453
Train Accuracy: 84.51%
Test Loss: 0.6701
Test Accuracy: 77.99%
-----
```

```
Epoch 10/15
Train Loss: 0.4028
Train Accuracy: 86.02%
Test Loss: 0.6689
Test Accuracy: 78.31%
-----
```

```
Epoch 11/15
Train Loss: 0.3750
Train Accuracy: 87.11%
Test Loss: 0.6689
Test Accuracy: 78.72%
-----
```

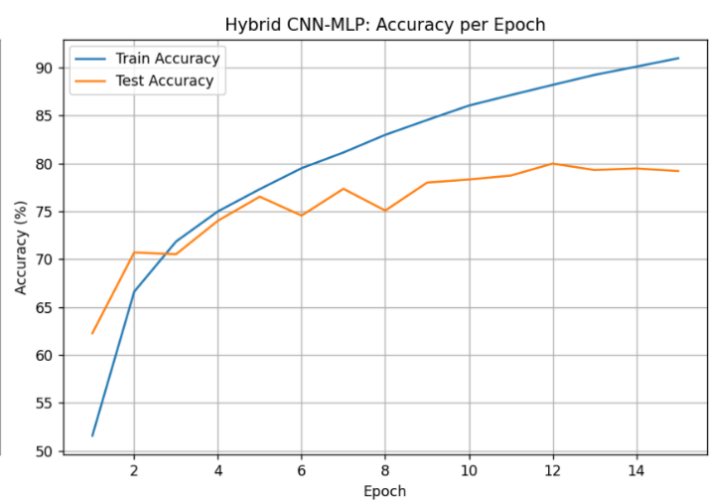
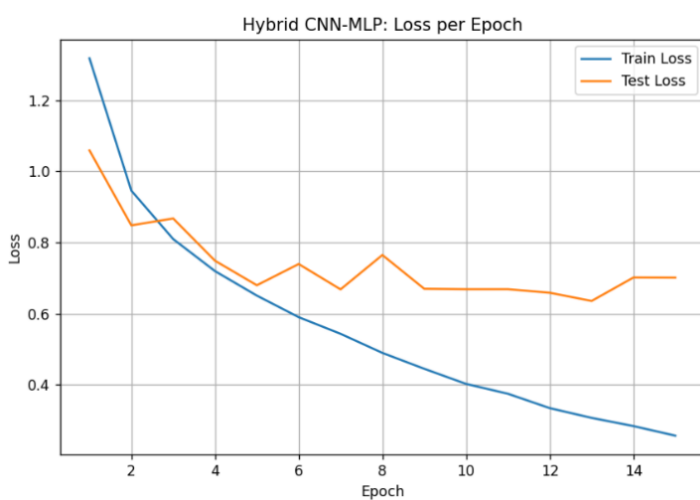
```
Epoch 12/15
Train Loss: 0.3346
Train Accuracy: 88.17%
Test Loss: 0.6589
Test Accuracy: 79.97%
-----
```

```
Epoch 13/15
Train Loss: 0.3072
Train Accuracy: 89.21%
Test Loss: 0.6358
Test Accuracy: 79.30%
-----
```

```
Epoch 15/15
Train Loss: 0.2572
Train Accuracy: 90.95%
Test Loss: 0.7014
Test Accuracy: 79.19%
-----
```

Final Summary

```
-----  
Hybrid Final Train Loss: 0.2572  
Hybrid Final Train Accuracy: 90.95%  
Hybrid Final Test Loss: 0.7014  
Hybrid Final Test Accuracy: 79.19%  
Best Hybrid Test Accuracy: 79.97%  
Training Time: 1981.49 seconds  
Number of Parameters: 1,276,682  
Estimated Model Memory Size: 4.870 MB
```



Model Comparison

Test Accuracy	Model	
53%	MLP	
64%	CNN	
79.97%	Hybrid CNN-MLP	

Conclusion

The Hybrid CNN-MLP model achieved the best overall performance among all evaluated architectures. While the MLP model achieved the lowest accuracy due to its inability to preserve spatial information, the CNN significantly improved performance by learning local image features. The proposed hybrid architecture combined the strengths of both approaches and achieved a best test accuracy of 79.97%, outperforming the standard CNN model (64%). Although some overfitting was observed, the hybrid network demonstrated strong classification performance and provided the best overall results on the CIFAR-10 dataset.

Hybrid CNN-MLP Architecture

